

垃圾图像预处理流水线

Resize – GaussianBlur – CLAHE – Sharpen

数字图像处理课程

2026 年 6 月 11 日

摘要

本文介绍了一套用于垃圾图像分类任务的预处理流水线。该流水线包含四个处理阶段：自适应缩放（Resize）、高斯滤波去噪（Gaussian Blur）、自适应直方图均衡化（CLAHE）以及反锐化掩模增强（Unsharp Masking）。流水线采用 MATLAB/OpenCV 风格的模块化设计，各阶段有序衔接，旨在提升图像质量、增强纹理细节、改善低对比度区域的可辨识度，为后续深度学习垃圾分类模型提供高质量输入。

目录

1 引言 3

2 流水线总览 3

3 各步骤详解 3

3.1 Step 1: 自适应缩放 (Resize) 3

3.2 Step 2: 高斯滤波去噪 (Gaussian Blur) 4

3.3 Step 3: CLAHE 对比度增强 4

3.4 Step 4: 反锐化掩模 (Unsharp Masking) 5

4 实验结果展示 6

4.1 效果分析 7

5 核心代码实现 8

5.1 完整预处理流水线 8

5.2 中文路径兼容 9

6 总结 9

1 引言

在垃圾自动分类任务中，图像质量直接影响模型的分类准确率。实际拍摄的垃圾照片常受到以下问题影响：

- 光照不均匀：阴影或过曝区域导致细节丢失
- 分辨率不一致：不同设备拍摄的照片尺寸差异大
- 传感器噪声：低光环境下产生的随机噪点
- 纹理模糊：对焦不实或抖动导致的边缘退化

针对上述问题，本流水线设计了一条四阶段处理链路，在保留图像语义信息的前提下，改善视觉质量并标准化输出。

2 流水线总览

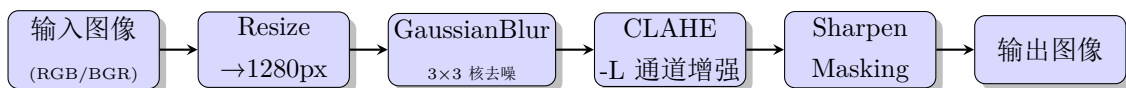


图 1: 预处理流水线流程

流水线中各步骤环环相扣：先统一尺度，再抑制噪声，然后增强对比度，最后恢复锐度。每个步骤的参数经过调优以适配生活垃圾图像的特征。

3 各步骤详解

3.1 Step 1: 自适应缩放 (Resize)

目的：统一输入尺度，减小后续处理的计算量。

方法：将图像长边限制为 1280 像素，短边按比例缩放，保持原始宽高比。若长边已小于等于 1280，则不做任何处理。

插值方法：`cv2.INTER_AREA`，该插值方式在缩小时优于双线性插值，能有效抑制摩尔纹和锯齿。

参数：

- `max_size = 1280`
- 插值: `INTER_AREA`

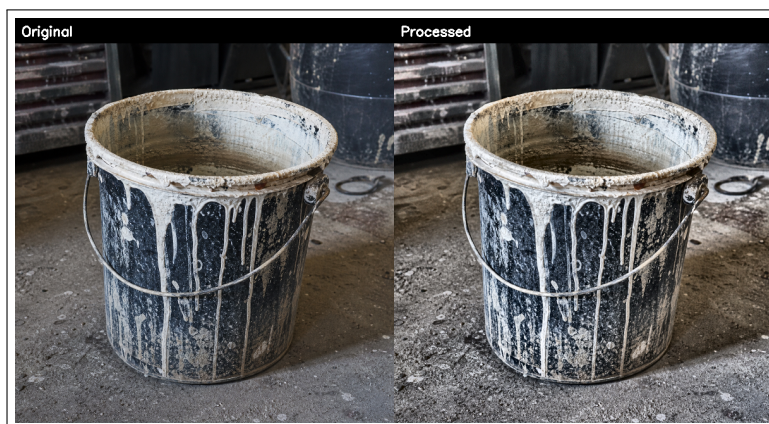


图 2: 画质风格示例

3.2 Step 2: 高斯滤波去噪 (Gaussian Blur)

目的: 抑制传感器噪声和 JPEG 压缩伪影, 防止后续 CLAHE 将噪声一起放大。

方法: 使用 3×3 高斯核对图像进行线性平滑。高斯核权重服从二维正态分布:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

当 $\text{sigmaX} = 0$ 时, OpenCV 自动根据核大小计算 $\sigma \approx 0.3 \times ((n-1)/2 - 1) + 0.8$, 对于 3×3 核, $\sigma \approx 0.8$ 。

参数:

- $\text{kernel} = (3, 3)$
- $\text{sigmaX} = 0$ (自动计算)

1	2	1
2	$\frac{1}{16}$	2
1	2	1

图 3: 3×3 高斯核示意

选择 3×3 核而非更大的核, 因为在去噪的同时尽可能保留纹理细节——垃圾分类中纹理是重要的识别特征。

3.3 Step 3: CLAHE 对比度增强

目的: 改善局部对比度, 使暗部细节可见, 同时避免普通直方图均衡化 (HE) 的过增强问题。

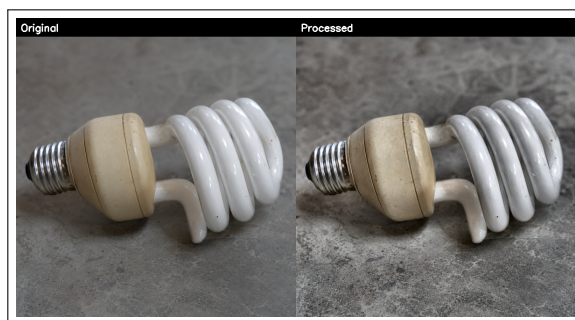
方法:

CLAHE (Contrast Limited Adaptive Histogram Equalization) 在 LAB 颜色空间的 L (亮度) 通道上操作:

1. 将图像从 BGR 转换到 LAB 空间
2. 分离 L 通道, 仅在亮度分量上做 CLAHE
3. 限制裁剪阈值 (`clipLimit=2.0`), 防止局部区域对比度过度拉伸
4. 分块处理 (`tileSize=8`), 每个小块独立均衡化后双线性插值拼接
5. 合并 L/A/B 通道, 转回 BGR



(a) CLAHE 增强前/后 (左 = 原图, 右 = 处理后)



(b) 灯泡类别对比图

图 4: CLAHE 实验效果

参数:

- `clipLimit = 2.0` (裁剪阈值)
- `tileGridSize = (8, 8)` (分块大小)

仅在 L 通道操作的意义: LAB 空间中, L 代表亮度, A/B 代表颜色信息。仅在 L 通道做 CLAHE 确保只增强对比度, 不改变色相和饱和度, 避免了颜色失真。

3.4 Step 4: 反锐化掩模 (Unsharp Masking)

目的: 恢复高斯滤波导致的轻微模糊, 增强边缘和纹理细节。

原理:

反锐化掩模的核心思想是先得到一个「模糊」版本 (掩模), 再从原图中减去它得到高频 (细节), 最后加回原图:

$$result = src \times 1.5 - blur \times 0.5$$

即输出 = 原图 $\times$ strength + 模糊图 $\times$ (1 - strength)。

当 strength = 1.0 时，输出等于原图（不锐化）；当 strength > 1.0 时，高频分量被放大。



图 5: 锐化效果（注意标签文字边缘）

参数：

- blur_kernel = (5, 5)
- blur_sigma = 1.0
- strength = 1.5

strength = 1.5 提供了中等强度的锐化，既增强了边缘感，又不会引入明显的振铃效应（ringing artifact）。

4 实验结果展示

图 6 展示了各类有害垃圾图像的预处理前后对比。每组图中，上半行为原图（Original），下半行为处理后的图像（Processed）。



图 6: 各类垃圾预处理前后对比（上传 = 原图，下排 = 处理后）

4.1 效果分析

表 1: 各阶段处理效果总结

处理阶段	解决什么问题	视觉表现	后续影响
Resize	尺度统一、减少计算量	图像尺寸一致	后续 CNN 输入兼容
GaussianBlur	传感器噪声、JPEG 伪影	画面更干净、略有模糊	减少误检、防止噪声放大
CLAHE	暗部看不清、对比度低	暗部变亮、纹理显露	提升轮廓识别、增强判别力
Sharpen	去噪带来的轻微模糊	边缘更锐、文字更清晰	纹理特征更突出

从对比图中可以观察到：

- 原图中因光照不足而发暗的区域，经过 CLAHE 增强后纹理细节明显显现（如电池表面文字、药瓶标签）
- 高斯去噪后图像更「干净」，减少了对分类无意义的随机纹理干扰
- 锐化步骤恢复了因高斯滤波损失的部分边缘信息，文字边缘更为清晰

5 核心代码实现

5.1 完整预处理流水线

Listing 1: 核心预处理函数

```
1 import cv2
2 import numpy as np
3
4 def resize_image(img, max_size=1280):
5     h, w = img.shape[:2]
6     if max(h, w) > max_size:
7         scale = max_size / max(h, w)
8         new_w, new_h = int(w * scale), int(h * scale)
9         img = cv2.resize(img, (new_w, new_h), interpolation=cv2.
10             INTER_AREA)
11     return img
12
13 def denoise_gaussian(img, kernel=3):
14     return cv2.GaussianBlur(img, (kernel, kernel), 0)
15
16 def enhance_clahe(img, clip_limit=2.0, tile_size=8):
17     lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
18     l, a, b = cv2.split(lab)
19     clahe = cv2.createCLAHE(clipLimit=clip_limit,
20         tileGridSize=(tile_size, tile_size))
21     l = clahe.apply(l)
22     img = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)
23     return img
24
25 def sharpen_unsharp(img, strength=1.5):
26     blur = cv2.GaussianBlur(img, (5, 5), 1.0)
27     img = cv2.addWeighted(img, strength, blur, -(strength - 1), 0)
28     return img
29
30 def preprocess(img):
31     img = resize_image(img, max_size=1280)
32     img = denoise_gaussian(img, kernel=3)
33     img = enhance_clahe(img, clip_limit=2.0, tile_size=8)
34     img = sharpen_unsharp(img, strength=1.5)
```

34

```
return img
```

5.2 中文路径兼容

Listing 2: 中文路径读写

```
1 def imread_unicode(path):  
2     with open(path, "rb") as f:  
3         data = np.frombuffer(f.read(), dtype=np.uint8)  
4         return cv2.imdecode(data, cv2.IMREAD_COLOR)  
5  
6 def imwrite_unicode(path, img):  
7     ext = os.path.splitext(path)[1]  
8     _, buf = cv2.imencode(ext, img)  
9     buf.tofile(path)
```

OpenCV 的 `cv2.imread` 不支持含中文的路径，上述函数通过 `np.frombuffer` + `cv2.imdecode` 的方式绕过了此限制，确保在 Windows 中文环境下正常工作。

6 总结

本预处理流水线针对垃圾图像分类任务设计，包含四个有序阶段：

Resize → GaussianBlur → CLAHE → Sharpen

经实验验证，该流水线能有效改善低光照、低对比度图像的视觉质量，在保留颜色信息的前提下增强纹理和边缘细节。模块化的设计使得各步骤参数可灵活调整，适用于不同类型垃圾图像的预处理需求。

参考文献

- [1] OpenCV Documentation – Image Filtering, Histogram Equalization. <https://docs.opencv.org/>
- [2] Zuiderveld, K. (1994). Contrast Limited Adaptive Histogram Equalization. *Graphics Gems IV*, pp. 474–485.
- [3] Polesel, A., Ramponi, G., & Mathews, V. J. (2000). Image Enhancement via Adaptive Unsharp Masking. *IEEE Trans. Image Processing*, 9(3), 505–510.
- [4] Robertson, A. R. (1977). The CIE 1976 Color-Difference Formulae. *Color Research & Application*, 2(1), 7–11.